

GCC/GFortran Optimization Guide

Leveraging `wspy` Hardware Counters & AMD IBS Profiling Data to Guide Compiler Flags

1. Introduction & Methodology

Modern C/C++ and Fortran compilers (such as GCC and GFortran 16.2.0) provide hundreds of optimization switches. Choosing the right switches manually can be overwhelming. Standard advice often stops at generic levels like `-O2` or `-O3`. However, hardware performance counters produced by `wspy` (and AMD Instruction Based Sampling, **IBS**) provide precise insight into microarchitectural bottlenecks—such as pipeline stalls, vector register usage, memory bandwidth saturation, and branch mispredictions.

This manual establishes a systematic mapping between `wspy` metrics and targeted GCC/GFortran optimization flags. By analyzing performance runs against these guidelines, software engineers can tailor flag selection to address verified hardware bottlenecks rather than relying on trial-and-error.

Optimization Strategy Rule of Thumb

First address system-level and memory layout bottlenecks (page faults, NUMA, cache misses), then move to instruction-level optimizations (vectorization, unrolling, branch layout), and finally consider microarchitecture-specific tuning flags.

2. Timing, Process Resources & System Metrics

`on_cpu, utime, stime, nvcs, nvcsw, nivcsw``on_cpu: 0.914 (# 21.94/24 cores)`

Microarchitectural Meaning: High `utime` relative to `stime` indicates a pure compute-bound workload in userspace. High `nvcsw` (voluntary context switches) or high `stime` indicates thread synchronization, I/O bottlenecks, or kernel system call overhead. High `nivcsw` (involuntary context switches) indicates thread oversubscription.

Relevant GCC Flags & Actions:

- `-fopenmp` / `-ftree-parallelize-loops=N` : If `on_cpu` is significantly below available core capacity on a multi-core system, evaluate multi-threading loops.
- `-fno-semantic-interposition` : Reduces PLT overhead and indirect calls in dynamic library symbols, reducing system overhead in userspace.
- For OpenMP/Fortran runtimes: Environment tuning (e.g., `OMP_PROC_BIND=true`, `OMP_PLACES=threads`) to reduce involuntary switches.

minflt, majflt, page faults

minflt: 32,803,243 (# 117.9k/sec)

Microarchitectural Meaning: Minor page faults occur when memory allocation triggers page-table population upon first write/read (demand paging). Heavy minor fault rates indicate high dynamic allocation/deallocation overhead and poor memory reuse.

Relevant GCC Flags & Actions:

- `-fprefetch-loop-arrays` : Helps schedule prefetching and reduces page boundary lookup stalls.
- Linker/Alloc Flags: Encourage static allocation, stack arrays, or memory pooling. Use custom allocators (e.g., `jemalloc` or huge pages via `madvise` /Transparent Huge Pages).
- For GFortran: `-fno-automatic` (where safe) or `-fmax-stack-var-size=N` to adjust heap vs. stack allocation threshold for large array temporaries.

3. Topdown Pipeline & Execution Metrics (Pass 0 & 1)

ipc (Instructions Per Cycle)

instructions: 22.27T | cycles: 20.26T | IPC: 1.10

Microarchitectural Meaning: Measures execution efficiency. Modern x86 cores can retire up to 4–6 instructions per cycle. An IPC < 1.0 indicates severe pipeline stalling (memory latency, dependency chains, or speculation loss). An IPC > 2.5 represents high compute efficiency.

Relevant GCC Flags & Actions:

- `-O2` vs `-O3` : If IPC is low (< 1.0), upgrading to `-O3` enables vectorization and loop transformations that increase instruction-level parallelism (ILP).
- `-march=native` / `-mtune=native` : Ensures instruction scheduling matches core pipeline pipelines, execution units, and latency characteristics.

backend & backend_memory vs backend_cpu

backend: 38.9% (memory: 41.0%, cpu: 4.3%)

Microarchitectural Meaning: High `backend_memory` indicates execution slots stalled waiting on load/store operations, cache misses, or DRAM latency. High `backend_cpu` indicates resource contention on execution units (e.g., ALU port saturation, long-latency floating-point dependency chains).

Relevant GCC Flags & Actions:

- If `backend_memory` is high (> 25-30%):
 - `-fprefetch-loop-arrays` : Emits software prefetch instructions to hide memory latency.
 - `-ftree-loop-optimize` , `-floop-interchange` , `-floop-strip-mine` , `-floop-block` (requires Graphite via `-fgraphite-identity`) : Reorders loop nests to improve spatial and temporal cache locality.
 - `-fopt-info-loop` : Inspects compiler reports to verify loop nest optimizations.
- If `backend_cpu` is high (> 15-20%):
 - `-funroll-loops` : Exposes more independent operations to pipeline execution ports.
 - `-ffast-math` / `-fassociative-math` : Reassociates floating-point expressions to break long scalar dependency chains.
 - `-sched-spec-load` : Enables speculative scheduling of loads to relieve execution port pressure.

frontend, frontend_latency & frontend_bandwidth

frontend: 30.3% (latency: 20.4%, bandwidth: 14.9%)

Microarchitectural Meaning: CPU core pipeline is hungry because fetch/decode cannot supply micro-ops fast enough. High `latency` points to instruction cache (iCache) / iTLB misses; high `bandwidth` points to instruction decoding bottlenecks or bloated code size.

Relevant GCC Flags & Actions:

- `-O0` or `-O2` (instead of aggressive `-O3` or excessive unrolling): Reduces binary size to keep hot loops inside the L1 iCache / opcache.
- `-fprofile-generate` / `-fprofile-use` (PGO): Reorders basic blocks and places cold code out-of-line, drastically improving instruction fetch efficiency and L1 iCache hit rate.
- `-falign-functions=N` , `-falign-loops=N` , `-falign-jumps=N` : Ensures function and loop entry points align on 16/32/64-byte boundaries.

speculation & branch mispredicts

speculation: 3.5% | branch miss: 4.26%

Microarchitectural Meaning: CPU pipeline slots wasted executing instructions that are discarded due to mispredicted branches or pipeline restarts. Branch miss rates > 2-3% cause significant performance degradation.

Relevant GCC Flags & Actions:

- `-fprofile-use` (PGO): Provides actual branch probabilities to the compiler, enabling branch direction optimization and `__builtin_expect` automation.
- `-fmove-loop-invariants`, `-ftree-switch-conversion`: Converts indirect branches or complex conditional branches into direct lookups or branchless execution.
- Conditional moves: GCC uses branchless instructions (`cmov`, `vblendv`) when branch mispredictions are high.

4. Cache, Op-Cache & TLB Metrics (Pass 2, 3, 5 & IBS)

icache & opcache miss rates

icache miss: 11.2% | opcache miss: 24.7%

Microarchitectural Meaning: High instruction cache / micro-op cache miss rates mean the processor continuously falls back to expensive instruction decoding and L2/L3 fetches.

Relevant GCC Flags & Actions:

- `-fprofile-use` (PGO): Absolutely vital for high icache/opcache miss rates. Moves rarely executed error-handling code into separate memory sections (hot/cold splitting).
- `-finline-limit=N` / `-fno-inline-functions`: Avoids over-inlining that inflates code footprint.
- `--param max-inline-insns-single=N`: Restricts single-function expansion size.

dTLB miss & iTLB miss rates

l1 dTLB: 5.956 / 1k inst | l2 dTLB: 0.360 / 1k inst

Microarchitectural Meaning: Data and Instruction Translation Lookaside Buffer misses indicate high virtual address translation overhead (address space fragmentation across many 4KB pages).

Relevant GCC Flags & Actions:

- Code/Data Alignment: `-falign-labels`, `-falign-loops`.
- OS / Allocation level: Configure Transparent Huge Pages (THP, 2MB/1GB pages) and use `libhugetlbfs` or explicit `mmap(MAP_HUGETLB)` in source code.
- Fortran: `-fno-realloc-lhs` (prevents implicit array reallocations that fragment heap memory).

L2 / L3 / DRAM Accesses & IBS Data Breakdown

L2 miss: 9.85% | IBS DRAM rate: 53.2%

Microarchitectural Meaning: High L2/L3 miss rates and a high IBS DRAM rate (> 30-50%) indicate that the workload's active data set exceeds cache capacities, causing execution to be bound by DRAM latency and bandwidth.

Relevant GCC Flags & Actions:

- `-fprefetch-loop-arrays` : Forces GCC to emit hardware/software prefetch instructions (e.g. `prefetcht0`).
- `--param prefetch-latency=N` : Sets expected memory latency (in cycles) for loop prefetching calculations (increase N if DRAM rate is very high).
- `-fgraphite-identity` , `-floop-nest-optimize` : Enforce loop tiling (blocking) to keep data tiles in L1/L2 caches.
- GFortran specific: Ensure array access matches column-major order (e.g. `A(i, j)` with inner loop on `i`).

5. Vectorization & Floating-Point Operations (Pass 7)

The floating-point breakdown counter measures vector SIMD width utilization per 1,000 instructions. Vector width selection is a critical decision point for high-performance computing.

FP Counter Density	Workload Characteristic	Recommended GCC / GFortran Options
Low FP Density (< 5-10 FP ops / 1k inst)	Integer / Control / Logic Heavy	• Do NOT force high vector widths. • Stick to baseline <code>-O2 -march=native</code> . • Avoid <code>-mprefer-avx512</code> as frequency downclocking / transition penalties outweigh vector gains.
Moderate FP Density (10 - 50 FP ops / 1k inst)	Mixed Scalar & Vector Workload	• <code>-mprefer-avx256</code>: Ideal target. Provides 256-bit SIMD efficiency without core frequency downclocking penalties. • <code>-ftree-vectorize</code>: Enabled by default at <code>-O3</code> or explicitly at <code>-O2</code> . • <code>-ffast-math</code> or <code>-fno-math-errno</code>: Allows vectorization of math loops.
High FP Density (> 50-100+ FP ops / 1k inst)	Compute-Dense / Matrix / PDE Solvers	• <code>-mprefer-avx512</code> or <code>-mprefer-vector-width=512</code> (on AVX-512 supported CPUs). • <code>-mavx512f -mavx512dq -mavx512vl</code> (or <code>-march=native</code>). • <code>-ffast-math -fassociative-math</code>: Breaks IEEE-754 strict ordering to allow wide vector lane accumulation. • <code>-mveclibabi=acml</code> / SVML: Uses vectorized math library routines for <code>sin</code> , <code>cos</code> , <code>exp</code> .

Case Study: Sample Data Analysis

In the attached sample `counters.txt` , float instruction density is **37.8 FP ops / 1,000 inst** (dominated by AVX-128 at 33.5/1k and AVX-256 at 4.35/1k, with almost zero AVX-512). This represents a **moderate FP workload** that is currently under-vectorized. Setting `-mprefer-avx256` together with `-ftree-vectorize` and `-ffast-math` is the primary recommendation to convert 128-bit operations into 256-bit AVX2 operations.

6. Master GCC/GFortran Optimization Decision Table

wspy / IBS Trigger	Threshold Signal	Targeted GCC / GFortran Flag	Expected Effect / Rationale
backend_memory	> 25% of slots	-fprefetch-loop-arrays --param prefetch-latency=300	Emits prefetch instructions for memory streams.
backend_memory + high DRAM	IBS DRAM > 40%	-fgraphite-identity -floop-nest-optimize	Enables loop blocking/tiling to keep working set in cache.
backend_cpu	> 15% of slots	-funroll-loops -ffast-math	Exposes independent ALU operations; breaks long serial dependency chains.
frontend / icache miss	icache miss > 5% or FE > 20%	-fprofile-generate / -fprofile-use (PGO)	Optimizes code layout, basic block grouping, and branch prediction.
branch miss / speculation	Miss rate > 3%	-fprofile-use , -fmove-loop-invariants	Eliminates indirect branches; aids compiler in conditional move generation.
FP Density (Moderate)	20 – 50 FP / 1k inst	-mprefer-avx256 -ftree-vectorize	Upgrades 128-bit vector loops to 256-bit without AVX-512 downclocking.
FP Density (High)	> 50 FP / 1k inst	-mprefer-avx512 -ffast-math -ftree-vectorize	Maximizes vector register width for compute-heavy SIMD kernels.
dTLB miss	> 2.0 / 1k inst	Linker: Huge Pages; GFortran: -fno-realloc-lhs	Reduces page table walk overhead by expanding page frame size.
Fortran Loop Overhead	High loop overhead	-fno-stack-arrays -fexternal-blas	Allocates temporary arrays on heap; redirects heavy matrix loops to BLAS.

7. Step-by-Step Optimization Workflow for Developers

```

+-----+
| STEP 1: Baseline Measurement |
| Run wspy and capture counters.txt & ibs.txt |
+-----+
      |
      v
+-----+
| STEP 2: Topdown Bottleneck Classification |
| Check Retire vs Frontend vs Backend vs Speculation |
+-----+
      |
      +-----+
      |         |         |
      v         v         v
[ Backend Bound ] [ Frontend / ICache ] [ High Speculation ]
- Check Memory vs CPU - Apply PGO (-fprofile) - Apply PGO
- Memory: Prefetch & Tile - Reduce unrolling - Select branchless
- CPU: Unroll & Fast-Math - Align loops - Simplify logic

```

```

      |               |               |
      +-----+-----+
      |               |
      |               v
+-----+
| STEP 3: Vectorization & SIMD Optimization |
| Evaluate Float per 1000 instructions in Pass 7 |
| - High FP (>50/1k): -mprefer-avx512 |
| - Moderate FP (10-50/1k): -mprefer-avx256 |
| - Low FP (<10/1k): Avoid forcing 512-bit vectorization |
+-----+
      |
      |               v
+-----+
| STEP 4: Iterative Re-measurement & Validation |
| Re-run wspy; compare IPC, cache miss rates, and total elapsed time |
+-----+

```